Quiz 10-5

Give a scenario where a plain BST may outperform an AVL tree in practice.

힌트 보기

개념 정리 · AVL의 비용과 보장

- **비용**: AVL은 삽입·삭제 시 균형 정보를 점검·갱신하는 **부기(bookkeeping) 오버헤드**가 든다. 균형이 깨진 경우에만 회전을 수행하며, 삽입에서는 단일·이중 회전으로 복원하고 삭제에서는 여러 조상에서 추가 회전이 필요할 수 있다.
- **보장**: 대가로 worst-case 높이 $O(\log n)$을 보장한다. 이 보장이 **별 가치 없는 상황**을 찾을 것.

강의 슬라이드 Ch10 p48 Quiz

정답 보기

풀이

① **갱신이 잦고 검색이 드문 워크로드.** 삽입·삭제가 빈번하더라도 키의 삽입 순서가 BST를 심하게 편향시키지 않는다면, 단순 BST는 균형 정보 갱신과 회전 비용이 없어 AVL보다 빠를 수 있다.

> 다만 AVL의 균형 보장은 검색뿐 아니라 삽입·삭제의 최악 시간도 $O(\log n)$으로 제한하므로, 갱신이 많다는 사실만으로 BST의 우세가 보장되지는 않는다.

② **n이 작을 때.** 일반 BST의 최악 높이는 $O(n)$, AVL의 최악 높이는 $O(\log n)$이다. 그러나 데이터가 적으면 실제 탐색 경로 길이의 차이가 작을 수 있다.

> 이 경우 균형 유지 오버헤드가 없는 단순 BST가 AVL보다 빠를 수 있다. 실제 우세 여부는 입력 순서와 구현에 따라 달라진다.

③ **키가 무작위 순서로 삽입될 때.** 서로 다른 n개 키를 균등 무작위 순서로 삽입해 만든 일반 BST의 기대 높이는 $O(\log n)$이다. 따라서 이 조건에서는 AVL과 같은 차수의 탐색 성능을 기대할 수 있고, 균형 유지 오버헤드가 없는 BST가 더 빠를 수 있다.

> 다만 기대 높이는 개별 실행의 높이를 보장하지 않으며, 삭제가 반복되는 워크로드까지 같은 결과가 자동으로 적용되지는 않는다.

최종 답

갱신 위주(키 삽입 순서가 심하게 편향되지 않는 경우) · 작은 n · 무작위 삽입 순서 워크로드.
이런 상황에서는 균형 유지 오버헤드가 없는 일반 BST가 AVL보다 빠를 수 있다.

한 줄 정리 · 최악 경우 높이 보장의 필요성이 작고 BST가 크게 편향되지 않는 워크로드에서는, AVL의 균형 유지 비용 때문에 일반 BST가 더 유리할 수 있다.